

Dossier De Conception (DDC)

du projet

Robot Mini-Sumo

Responsabilité documentaire

Action	NOM Prénom	Fonction	Date	Signature
Rédigé par	Johan Baric–Monzat, Pascoal Dias, Clément Sancey, Flavio Dupuped, Joris Coppi, William Moreau	Technicien	26/09/25	
Approuvé par	<Chef de Projet> (IUT GEII Bdx)	Chef de projet	26/09/25	
Approuvé par	<Client> (IUT GEII Bdx)	Client	26/09/25	

Robot Mini-Sumo (RMS)

Suivi des révisions documentaires

Indice	Date	Nature de la révision
1	01/09/2022	Publication préliminaire du DDC document à compléter par le Technicien.
2	06/10/2025	Première publication

Documents de références

Sigle	Référence	Titre	Rév.	Origine
[CDC]	RMS_CDC	Cahier des charges	1	IUT GEii Bdx

/* Partie à supprimer avant envoi au client
en vert : les parties du rapport à modifier

en jaune : les consignes de rédaction

en bleu : les parties rédigées par M. Blanchard

*/

Table des matières

1. Nature du document	6
2. Conception préliminaire du produit	6
2.1 Architecture Électronique	6
2.1.1 Choix des capteurs adversaire	10
2.1.2 Choix des capteurs de sol	12
2.1.3 Choix du pont en H	15
2.1.4 Choix du Motoréducteur	16
2.1.5 Choix de la batterie	18
2.1.6 Choix du régulateur de tension	19
2.1.7 Choix de la Méthode sécurité de la batterie	19
2.1.8 Traitement de l'information	21
2.2 Architecture Informatique	23
2.2.1 Fonctions d'acquisition	24
2.2.2 Fonctions d'action	25
2.2.3 Fonctions d'énergie	27
2.2.4 Fonctions de traitement	27
2.3 Conclusion de la conception préliminaire du produit	29
3. Conception détaillée du produit	31
3.1 Conception détaillée du robot mini-sumo	31
3.2 Conception détaillée de la partie acquisition	32
3.2.1 Capteurs adversaires	32
3.2.1.1 Schéma électrique des capteurs adversaires	32
3.2.1.2 Code informatique des capteurs adversaires	33
3.2.2 Capteurs de sol (si nécessaire)	34
3.2.2.1 Schéma électrique des capteurs de sol	34
3.2.2.2 Code informatique des capteurs de sol	35
3.2.3 Capteurs de télécommande (reprendre le DDF de M. Blanchard)	36
3.2.3.1 Schéma électrique des capteurs de télécommande	36
3.2.3.2 Code informatique des capteurs adversaires	36
3.3 Conception détaillée de la partie action	38
3.3.1 Pont en H	38
3.3.1.1 Schéma électrique du pont en H	38
3.3.1.2 Code informatique du pont en H	38
3.4 Conception détaillée de la partie énergie	42
3.4.1 Pont diviseur de tension	42
3.4.1.1 Schéma électrique du pont diviseur de tension	43
3.4.1.2 Code informatique du pont diviseur de tension	43
3.4.2 régulateur de tension	43

3.4.2.1 Schéma électrique du régulateur de tension	44
3.4.3 Autonomie	44
3.4.3.1 Schéma électrique	44
3.5 Conception détaillée de la partie traitement	44
3.5.1 Le microcontrôleur ATMEGA328P-PU	44
3.4.1.1 Schéma électrique du microcontrôleur	45
3.3.1.2 Code informatique du microcontrôleur	45
4. Dérisquage des solutions techniques retenues	46
4.1 <Titre de la simulation / prototypage rapide>	46
4.2 <Titre de la simulation / prototypage rapide>	47
4.3 Conclusion de la simulation / prototypage rapide du produit	49
5. Conclusion de la conception du produit	50
6. Matrice de conformité du produit	50

1. Nature du document

Ce document est un dossier de conception et a pour but de détailler la conception du produit développé. Il apporte ainsi des preuves de la conformité du produit par rapport à l'ensemble des exigences client. Le paragraphe 3 du [CDC] décrit de façon plus détaillée la nature et le positionnement de ce document dans l'arborescence documentaire du projet.

2. Conception préliminaire du produit

Ce chapitre décrit l'architecture fonctionnelle du produit. Il apporte les premiers éléments de preuve de la faisabilité du produit vis-à-vis des exigences client.

2.1 Architecture Électronique

Référence de pré-conception : CPR 1

Exigences client vérifiées par préconception :
EXIG_CHASSIS_DIMENSIONS, EXIG_INTEGRITE_MECA,
EXIG_FIXATION_CARTE, EXIG_AUTONOMIE, EXIG_SECUR_BATT, EXIG_ADVERSAI
RE, EXIG_LUMINOSITE, EXIG_DEPART, EXIG_DEPLACEMENT, EXIG_COMPORTEM
ENT, EXIG_CARTE

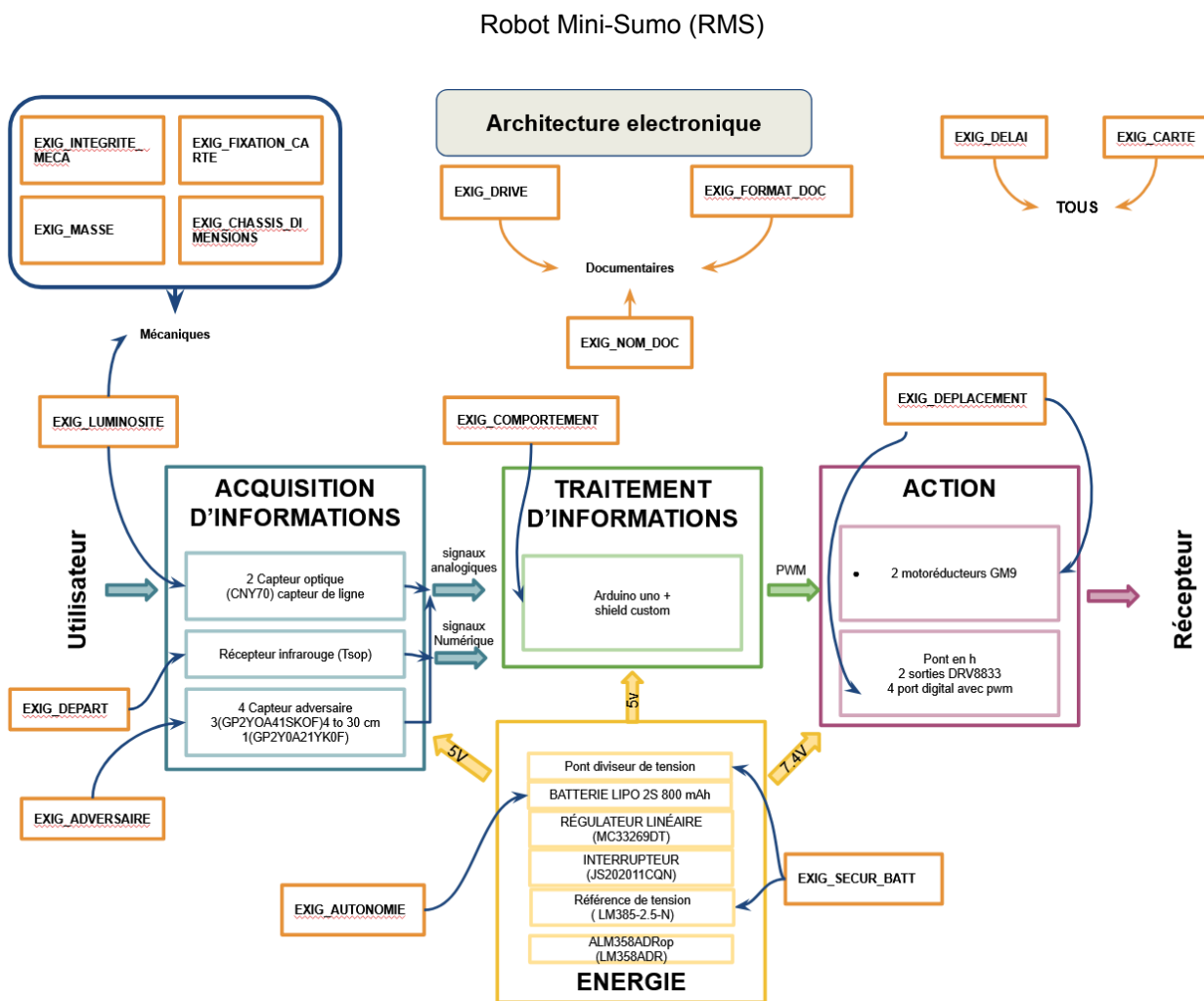


Figure 1 : Architecture électronique du robot sumo

Afin de répondre au cahier des charges, une analyse globale des exigences a conduit à l'architecture fonctionnelle présentée ci-dessous.

Voici la composition de chaque bloc :

Bloc Acquisition d'informations

Ce bloc constitue la partie du système dédiée à la perception de l'environnement et à la collecte de données nécessaires au fonctionnement global. Il repose sur un ensemble de capteurs permettant d'interpréter la situation autour du dispositif. Deux capteurs optiques de type CNY70 assurent la détection de lignes au sol, facilitant par exemple le suivi d'une trajectoire prédéfinie. Un récepteur infrarouge TSOP est utilisé pour recevoir des signaux provenant d'une télécommande ou d'un autre système émetteur, tandis que plusieurs capteurs de distance infrarouges (GP2Y0A41SK0F et GP2Y0A21YK0F) détectent la présence d'obstacles ou d'adversaires dans un rayon d'environ trente centimètres.

Ces capteurs transmettent des signaux analogiques et numériques vers le module de traitement d'informations. Les premiers traduisent des valeurs continues telles que la luminosité ou la distance mesurée, tandis que les seconds indiquent des états binaires, comme la détection ou non d'un obstacle. Ce bloc doit répondre à plusieurs exigences : la prise en compte de la luminosité ambiante, la gestion du départ du système, la reconnaissance d'un adversaire et la garantie d'une autonomie suffisante.

Bloc Énergie

Le bloc Énergie assure l'alimentation électrique de l'ensemble du dispositif. Il a pour rôle de fournir une tension stable et adaptée à chaque composant tout en garantissant la sécurité de l'alimentation. Le système repose sur une batterie Li-Po 2S de 800 mAh délivrant une tension nominale de 7,4 volts. Cette tension est ensuite régulée par un composant linéaire MC33269DT pour obtenir une alimentation stable de 5 volts, nécessaire au fonctionnement des capteurs et du microcontrôleur.

Un interrupteur permet d'activer ou de couper l'alimentation générale, tandis qu'un pont diviseur de tension et une référence LM385-2.5-N assurent le suivi précis du niveau de charge. Un amplificateur opérationnel LM358ADR complète l'ensemble en stabilisant le signal électrique. Ce bloc veille donc à alimenter le système de manière fiable, tout en répondant aux exigences d'autonomie énergétique et de sécurité de la batterie, notamment contre les risques de surcharge ou de décharge profonde.

Bloc Action

Le bloc Action correspond à la partie motrice du système, celle qui exécute les décisions issues du traitement d'informations. Il est constitué principalement de deux motoréducteurs GM9, responsables du déplacement du robot, commandés par un pont en H DRV8833. Ce dernier permet de contrôler à la fois la direction et la vitesse des moteurs grâce à la modulation de largeur d'impulsion (PWM).

L'ensemble reçoit les signaux de commande en provenance du bloc de traitement et les traduit en actions mécaniques précises. Ce module répond aux exigences de déplacement définies pour le système, en garantissant un mouvement fluide, réactif et sûr, tout en préservant la stabilité de l'alimentation électrique des moteurs.

Bloc Traitement d'information

Le bloc Traitement d'informations constitue le cœur logique et décisionnel du système. Il est chargé d'interpréter les données reçues des capteurs, de les analyser, puis de générer les ordres de commande nécessaires à l'action. Le dispositif repose sur une carte Arduino Uno, épaulée par un shield personnalisé qui permet d'adapter les connexions et d'intégrer des fonctionnalités spécifiques au projet.

Les signaux analogiques et numériques issus du bloc d'acquisition sont traités selon le programme embarqué sur la carte. Ce traitement permet de déterminer les actions à entreprendre, qui sont ensuite transmises au bloc d'action sous forme de signaux PWM. Ce module répond à l'exigence de

Robot Mini-Sumo (RMS)

comportement global du système, en traduisant les informations perçues en réactions adaptées, garantissant ainsi la cohérence et la réactivité de l'ensemble.

2.1.1 Choix des capteurs adversaire

Référence de pré-conception : CPR 2

Exigences client vérifiées par préconception : [EXIG_ADVERSAIRE](#)

FICHE D'AIDE A LA DECISION (FAD)									
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)						Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Distance min et max de détection	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité		
Capteur de mesure Sharp GP2Y0A41SK0F	0	min : 4 cm max : 30 cm		5V Typ	12mA Typ	15,42TTC	OUI	0	
		0	5	5	5	3	5		
Capteur de mesure Sharp GP2Y0A021YK	1	min : 10 cm max : 80 cm		5V Typ	30mA Typ	13,73TTC	OUI	12500	
		5	5	5	4	5	5		
Capteur de mesure Sharp GP2Y0A02YK	1	min : 20 cm max : 150 cm		5V Typ	33mA Typ	17,24TTC	OUI	375	
		1	5	5	3	1	5		
IUT de Bordeaux	Référence : SUMO_FAD								3 / 7
Département GEII	Révision : 1.6 – 19/09/2022								

Figure 2 : FAD pour le choix des capteur avant

Pour répondre aux exigences et choisir le capteur le plus approprié à notre projet, nous avons rempli des FAD (fiches d'aide à la décision).

Ces fiches nous permettent d'établir un score selon plusieurs paramètres importants, définis au préalable en fonction de notre utilité, et en les multipliant entre eux. Nous avons décidé de choisir un de ces capteur infrarouge pour notre robot sumo à cause des perturbations que peuvent avoir les capteurs à ultrasons. ([Liens vers les FAD](#))

Pour nos capteurs avant, nous en avons retenu trois, et cette FAD nous a permis de trancher afin de choisir celui qui correspondait le mieux à nos différentes exigences.

Le premier paramètre est la distance maximale et minimale de détection de chacun des capteurs. La différence est évaluée par rapport à la taille de l'arène de combat et à nos exigences.

L'exigence [EXIG_ADVERSAIRE](#) stipule que le robot mini-sumo doit être capable de détecter si son adversaire est devant lui à une distance minimale de 40 cm. Cette contrainte nous a permis d'écarter le premier capteur, qui ne détecte qu'à une distance maximale de 30 cm, car il ne la satisfait pas.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	10/51
----------------------------------	--	-------

L'arène mesure 77 cm de diamètre. Concernant le capteur 3, nous avons choisi de lui attribuer la valeur 0, car l'objectif de notre robot est d'être le plus rapide possible. En effet, comme l'arène ne fait que 77 cm de diamètre, ce capteur détectera beaucoup trop loin, ce qui compliquerait inutilement le traitement des données. Nous serions alors contraints de bloquer toutes les valeurs supérieures à 80 cm pour éviter une mauvaise interprétation.

Le paramètre suivant est la facilité d'utilisation du signal de sortie. Par cela, nous entendons la simplicité de traitement des données reçues par le capteur. Nous avons pu tester chacun des capteurs, et pour le traitement des données, le résultat est le même pour les trois : ils sont relativement simples à exploiter. Nous avons donc décidé de leur attribuer à tous la note de 5.

Pour la compatibilité de la tension avec les sources disponibles, nous nous sommes aidés des datasheets afin de déterminer la tension nécessaire pour chacun des capteurs. Nous pouvons constater que, de manière typique (valeur moyenne), chacun des capteurs est alimenté en 5 V. Cette valeur est parfaitement adaptée à notre circuit, qui fonctionne lui aussi en 5 V. Cela nous évite donc d'ajouter un système d'adaptation de tension. Nous avons ainsi attribué la note de 5 aux trois capteurs.

Nous avons également étudié, dans ces FAD, la consommation en courant de ces capteurs. Pour cela, nous avons travaillé avec les coéquipiers en charge de l'énergie, en prenant en compte les calculs de consommation totale de courant. La consommation des capteurs n'est pas un critère déterminant, car même avec celui qui consomme le plus (33 mA typiques), nous restons largement dans les limites supportées par la batterie. Nous avons néanmoins établi une hiérarchie décroissante, allant du capteur qui consomme le moins en énergie à celui qui consomme le plus.

Pour le paramètre du prix, nous avons pu chercher les prix des composants sur farnell. Nous pouvons voir que sur l'exigence du prix le moins chère est le deuxième capteur. Et pour finir l'ensemble de ces 3 capteurs sont actuellement accessibles dans nos stocks.

Pour conclure, nous constatons que le capteur infrarouge numéro 1 obtient la note de 0, car il ne respecte pas une exigence du cahier des charges. Le troisième capteur présente une note relativement basse de 375 : il répond à plusieurs de nos paramètres, mais pas de manière pleinement satisfaisante. En revanche, le deuxième capteur atteint un score de 12 500, nettement supérieur aux autres. Nous avons donc décidé de retenir ce capteur pour notre robot.

Nous utiliserons ainsi deux capteurs infrarouge Sharp GP2Y0A021YK, qui seront placés à l'arrière du robot mais orientés vers l'avant.

2.1.2 Choix des capteurs de sol

Référence de pré-conception : CPR 3

Exigences client vérifiées par préconception : [EXIG_CARTE](#)

FICHE D'AIDE A LA DECISION (FAD)								
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)					Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Distance min et max de détection	Compatibilité de la tension avec les sources disponibles	Respect de la stratégie de combat	Faible Prix	Disponibilité		
0 CNY avant + 0 CNY arrière	1	/5mm	1,25V à 1,6V	NON	0€	OUI	625	
		5	5	1	5	5		
1 CNY avant + 0 CNY arrière	1	/5mm	1,25V à 1,6V	NON	0,71€	OUI	625	
		5	5	1	5	5		
2 CNY avant + 0 CNY arrière	1	/5mm	1,25V à 1,6V	NON	1,42€	OUI	1875	
		5	5	3	5	5		
0 CNY avant + 1 CNY arrière	1	/5mm	1,25V à 1,6V	NON	0,71€	OUI	625	
		5	5	1	5	5		
1 CNY avant + 1 CNY arrière	1	/5mm	1,25V à 1,6V	NON	1,42€	OUI	625	
		5	5	1	5	5		
2 CNY avant + 1 CNY arrière	1	/5mm	1,25V à 1,6V	NON	2,13€	OUI	1500	
		5	5	3	4	5		
2 CNY avant + 2 CNY arrière	1	/5mm	1,25V à 1,6V	OUI	2,84€	OUI	2500	
		5	5	5	4	5		
IUT de Bordeaux	Référence : SUMO_FAD							4 / 7
Département GEII	Révision : 1.6 – 19/09/2022							

Figure 3 : FAD pour le choix des capteur de sol

https://docs.google.com/spreadsheets/d/1_x7JtwGLK-ZNtRol54Q7alMxSsFxZRl2iP1DH9i36V4/edit?usp=drive_link

Pour sélectionner la configuration optimale de capteurs CNY pour notre robot sumo, nous avons élaboré une fiche d'aide à la décision (FAD) permettant d'évaluer différentes combinaisons possibles. Cette approche méthodique nous permet d'attribuer un score à chaque solution technique en fonction de plusieurs critères essentiels, préalablement définis selon nos besoins spécifiques. Les capteurs CNY sont des capteurs infrarouges réfléchifs qui nous permettront de détecter les lignes blanches délimitant l'arène de combat. ([Liens vers les FAD](#))

Nous avons analysé sept configurations différentes, combinant des capteurs CNY à l'avant et à l'arrière du robot (de 0 à 2 capteurs pour chaque position). Le premier critère évalué est la conformité à toutes les exigences du cahier des charges. Ce paramètre est binaire : une configuration obtient 1 si elle répond à toutes les exigences, et 0 dans le cas contraire. Nous constatons que toutes les configurations testées satisfont ce critère de base, chacune obtenant donc la note de 1.

Le deuxième paramètre concerne la distance minimale et maximale de détection. Toutes les configurations testées présentent une distance de détection de 5 mm, ce qui est parfaitement adapté à la détection des lignes blanches au ras du sol. Nous avons donc attribué la note de 5 à l'ensemble des solutions pour ce critère.

Pour la compatibilité de la tension avec les sources disponibles, nous avons consulté les caractéristiques techniques. Les capteurs CNY fonctionnent avec une tension comprise entre 1,25 V et 1,6 V, compatible avec notre alimentation. Toutes les configurations reçoivent donc la note de 5 sur ce paramètre, évitant ainsi l'ajout d'un système d'adaptation de tension supplémentaire.

Concernant le respect de la stratégie de combat, nous avons établi une hiérarchie en fonction du nombre de capteurs et de leur positionnement. Les configurations avec aucun ou un seul capteur (0 CNY avant + 0 CNY arrière, 1 CNY avant + 0 CNY arrière, 0 CNY avant + 1 CNY arrière, et 1 CNY avant + 1 CNY arrière) obtiennent la note de 1, car elles offrent une couverture limitée. La configuration avec 2 CNY avant et 0 CNY arrière atteint un score de 3, représentant une amélioration significative pour la détection frontale. Les configurations avec 2 CNY avant et 1 CNY arrière ainsi que 2 CNY avant et 2 CNY arrière obtiennent respectivement les notes de 3 et 5, cette dernière offrant la couverture la plus complète pour notre stratégie de combat.

Le paramètre de fiabilité et de facilité de consommation du courant a également été évalué. Toutes les configurations obtiennent la note de 5 sur ce critère, ce qui indique que même avec le nombre maximal de capteurs, la consommation reste largement acceptable par rapport aux capacités de notre batterie. Nous restons donc dans les limites supportées par notre système d'alimentation.

Enfin, tous les capteurs CNY sont disponibles dans nos stocks, ce qui facilite l'approvisionnement et réduit les délais de réalisation. Chaque configuration reçoit donc la note de 5 pour le critère de disponibilité.

En conclusion, nous observons une progression nette des scores en fonction du nombre et du positionnement des capteurs. Les configurations les plus simples (0 CNY avant + 0 CNY arrière et 1 CNY avant + 0 CNY arrière) obtiennent un score de 625, insuffisant pour nos besoins. La configuration avec 2 CNY avant et 0 CNY arrière atteint 1 875 points, montrant une amélioration notable. Les configurations intermédiaires (0 CNY avant + 1 CNY arrière et 1 CNY avant + 1 CNY arrière) obtiennent également 625 points. La configuration avec 2 CNY avant et 1 CNY arrière atteint 1 500 points. Finalement, la configuration avec 2 capteurs CNY à l'avant et 2 capteurs CNY à l'arrière obtient le score maximal de 2 500, nettement supérieur aux autres solutions. Nous avons donc retenu cette configuration pour notre robot. Elle nous permettra de détecter efficacement les

Robot Mini-Sumo (RMS)

lignes blanches de l'arène, quelle que soit l'orientation du robot, garantissant ainsi qu'il reste dans la zone de combat tout en optimisant notre stratégie offensive et défensive.

2.1.3 Choix du pont en H

Référence de pré-conception : CPR 4

Exigences client vérifiées par préconception : [EXIG_DEPLACEMENT](#), [EXIG_CARTE](#)

FICHE D'AIDE A LA DECISION (FAD)											
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)								Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Compatibilité de la tension avec les sources disponibles	Courant de peak du pont	Dimensions	Courant max de sortie du pont	Manière de piloter le pont	Faible Prix	Disponibilité			
Pololu Commande de 2 moteurs CC DRV8835 2x1,2A	0	oui	1,5 A	18,56 x 10,46 x 0,46	1,2 A	deux moteurs CC	7,70 €	oui	46875	Après le test, on a constaté que ce moteur est plus vite	
		5	3	5	5	5	5	5			
Pololu Commande de 2 moteurs CC DRV8833 2x1,2A	1	oui	2 A	21 x 13 x 3	1,2 A	deux moteurs CC	11,40 €	oui	37500		
		5	5	4	5	5	3	5			
IUT de Bordeaux	Référence : PRJ_FAD									5 / 7	
Département GEII	Révision : 1.6 – 19/09/2022										

Figure 4 : FAD pour le choix du pont en H

Choix du pont en H

Pour répondre aux exigences et choisir le pont en H le plus adapté à notre projet, nous avons rempli des FAD (fiches d'aide à la décision). Ces fiches nous permettent d'évaluer les composants selon plusieurs paramètres importants, définis en fonction de notre utilisation, et de les comparer afin de retenir celui qui répond le mieux aux exigences du cahier des charges. ([Liens vers les FAD](#))

Pour piloter nos deux moteurs à courant continu, nous avons comparé deux composants : le Pololu DRV8833 et le DRV8835. Ces deux circuits présentent de fortes similitudes : ils permettent tous deux de contrôler deux moteurs indépendamment et sont compatibles avec les tensions d'alimentation disponibles.

Le premier paramètre que nous avons évalué est le courant maximal et de crête. Le DRV8833 supporte un courant de crête de 2 A tandis que le DRV8835 supporte 1,5 A. La consommation de nos moteurs étant inférieure à 1 A, les deux composants sont suffisants pour assurer leur fonctionnement. Cependant, des tests pratiques avec nos moteurs et mesures au tachymètre ont montré que le DRV8833 permettait d'atteindre des vitesses maximales plus élevées, ce qui constitue un avantage pour la performance du robot.

Le deuxième paramètre est la taille et l'intégration sur la carte. Le DRV8833 est plus grand que le DRV8835, mais reste compatible avec l'espace disponible sur notre carte électronique. La différence de taille n'est donc pas un critère limitant pour notre projet.

Robot Mini-Sumo (RMS)

Le troisième paramètre est le prix. Le DRV8833 est plus coûteux que le DRV8835, mais cette différence est compensée par l'amélioration des performances et la vitesse maximale atteinte.

Pour conclure, même si les deux composants peuvent alimenter nos moteurs, le DRV8833 a été retenu afin de garantir une vitesse maximale supérieure et de répondre pleinement aux exigences :

- [EXIG_DEPLACEMENT](#) : contrôle efficace et indépendant des deux moteurs à courant continu
- [EXIG_CARTE](#) : intégration d'un pont en H sur la carte électronique

2.1.4 Choix du Motoréducteur

Référence de pré-conception : CPR 5

Exigences client vérifiées par préconception : [EXIG_DEPLACEMENT](#)

FICHE D'AIDE A LA DECISION (FAD)									
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (valeur de 1 à 5)						Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Compatibilité de la tension avec les sources disponibles	Unloaded Current	peak current	Vitesse	Faible Prix	Disponibilité		
GM3		oui	40	n/a	46	6,9	oui	1250	
		5	5	2	1	5	5		
GM9		oui	52	700	78rpm	6,9	oui	1500	
		5	3	4	5	5	5		
IUT de Bordeaux	Référence : PRJ_FAD								2 / 7
Département GEII	Révision : 1.6 – 19/09/2022								

Figure 5 : FAD pour le choix du Motoréducteur

Choix des motoréducteurs

Afin de répondre aux exigences du cahier des charges concernant le déplacement du robot, nous avons réalisé une FAD (Fiche d'Aide à la Décision) pour choisir le moteur le plus adapté parmi ceux disponibles en stock.

Deux motoréducteurs ont été comparés : le **GM3** et le **GM9**, tous deux compatibles mécaniquement avec les roues **GMPW** imposées. Ces moteurs permettent une fixation simple sur le châssis grâce au support **RM3**.

Paramètres évalués :

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	16/51
----------------------------------	--	-------

([Liens vers les FAD](#))

1. Compatibilité de la tension d'alimentation

Les deux motoréducteurs fonctionnent avec une tension nominale de 6 V et peuvent être utilisés jusqu'à 9 V selon leurs fiches techniques. Ils sont donc parfaitement compatibles avec la batterie LiPo 2S (7,4 V) prévue, sans qu'il soit nécessaire d'ajouter un régulateur de tension.

2. Courant à vide et courant de crête

Le GM3 présente un courant à vide d'environ 40 mA, tandis que le GM9 consomme 52 mA. Le courant de crête du GM9 atteint environ 700 mA, mais reste compatible avec le pont en H sélectionné.

3. Vitesse de rotation

Le GM3 tourne à 46 tr/min, tandis que le GM9 atteint 78 tr/min. Le GM9 permet donc d'obtenir un déplacement plus rapide du robot, répondant mieux à l'exigence [EXIG DEPLACEMENT](#) du cahier des charges.

4. Prix et disponibilité

Les deux motoréducteurs sont proposés à un prix similaire (6,90 €) et sont disponibles en stock, ne créant ainsi aucune contrainte d'approvisionnement.

Analyse et choix

Les résultats de la FAD montrent que le GM9 obtient la meilleure note globale grâce à sa vitesse supérieure, tout en restant compatible avec la tension d'alimentation et le pont en H choisi. Ce moteur offre un compromis idéal entre rapidité et consommation

Nous avons donc décidé d'utiliser deux motoréducteurs GM9 couplés à deux roues GMPW malgré le fait que le moteur GM3 soit imposée dans le cahier des charges par l'exigence [EXIG CARTE](#) car sur le châssis du robot fourni des GM9 sont monté il y donc surement une erreur dans le cahier des charges qu'il faudra corriger.

Exigences satisfaites :

- [EXIG DEPLACEMENT](#) : assurer la mobilité et le contrôle du robot.
- [EXIG CARTE](#) : compatibilité électrique et mécanique avec le pont en H sélectionné.

t2.1.5 Choix de la batterie

Référence de pré-conception : CPR 6

Exigences client vérifiées par préconception : [EXIG_AUTONOMIE](#)

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)				Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Encombrement	Autonomie	Faible Prix	Disponibilité		
Lipo CONRAD ENERGY 7.4 V 1200 mAh	0	Longueur 72 mm; Largeur 36 mm; Hauteur 12 mm	614mah	€17,99	En stock	0	Plus encombrante que la limite fixée dans le cahier des charges
Lipo CONRAD ENERGY 7.4 V 800 mAh	1	Longueur 56 mm; Largeur 31 mm; Hauteur 17 mm	614mah	€12,99	En stock	375	
Lipo CONRAD ENERGY 7.4 V 450 mAh	0	Longueur : 56 mm; Largeur : 31 mm; Hauteur 11.5 mm	614mah	€8,29	En stock	0	Autonomie trop faible
IUT de Bordeaux	Référence : SUMO_FAD						
Département GEII	Révision : 1.6 – 19/09/2022						7 / 7

Figure 6 : FAD pour le choix de la batterie

Nous avons sélectionné la batterie LiPo CONRAD ENERGY 7.4 V 800 mAh. En effet, la batterie de 1200 mAh est plus encombrante que la limite fixée par le cahier des charges. La batterie de 450 mAh ne permet pas une autonomie suffisante. Nous avons donc sélectionné la batterie 800 mAh, en effet, elle respecte l'exigence de dimension fixée dans le cahier des charges. Elle respecte également l'exigence d'autonomie, elle permet un fonctionnement continu de 65 min sans obstacle (correspondant à un combat de 15 min).([Liens vers les FAD](#))

Afin de calculer l'autonomie, nous avons sommé les consommations de tous les composants :

- 2 motoréducteurs (133.92mA)
- 4 CNY70 (300mA)
- 2 GP2Y0A21YKOF (60 mA)
- 1 Arduino (50mA)
- 2 DRV 8833 (3.4 mA)
- 1 récepteur infrarouge (3 mA)

Total : 550.32 mA

Nous calculons alors la capacité minimale de la batterie : $E = i \times t$

$$E = 550.32 \times 1.08 = 594.3 \text{ mAh}$$

Nous prenons une marge de 20% donc $E = 594.3 \times 1.2 = 713.6 \text{ mAh} < 800 \text{ mAh}$

2.1.6 Choix du régulateur de tension

Référence de pré-conception : CPR 7

Exigences client vérifiées par préconception : [EXIG_SECUR_BATT](#)

Afin d'alimenter cette carte, nous avons été obligés d'utiliser un régulateur linéaire 5 V (figure xx). Pour sélectionner le bon régulateur, nous avons réalisé une FAD :

FICHE D'AIDE A LA DECISION (FAD)										
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)							Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Tension max d'entrée	Tension de sortie	Faible chute de tension sortie-entrée	Courant de sortie max	Boîtier compact facile à utiliser/soudier	Faible Prix	Disponibilité		
LP2985AIM5-3.3/NOPB 3.3V CMS	0	16 V.	3,3v	300-mV	150mA	oui	0,8	oui	0	
		5	0	3	2	5	4	5		
LP2985AIM5-5.0/NOPB 5V CMS	1	16 V.	5v	300-mV	150mA	oui	0,8	oui		
		5	5	5	1	5	4	5	12500	
MC33269DT-3.3G IC	0	20	3,3v	1v	800mA	oui	0,60 €	oui		
		5	0	2	5	5	5	5	0	
MC33269DT-5.0G IC	1	20	5v	1v	800mA	oui	0,60 €	oui		
		5	5	3	5	5	5	5	46875	

Figure 7 : FAD pour le choix du régulateur linéaire

Les critères de sélection majeurs étaient la tension de sortie de 5 V et la tension d'entrée devant pouvoir supporter 7,4 V. De plus, la consommation moyenne de la carte Arduino est d'environ 270mA, en raison du grand nombre de capteurs sélectionnés. ([Liens vers les FAD](#))

Ces éléments nous ont conduits à choisir le régulateur MC33269DT-5.0G, car il était le plus adapté à nos besoins.

2.1.7 Choix de la Méthode sécurité de la batterie

Référence de pré-conception : CPR 8

Exigences client vérifiées par préconception : [EXIG_SECUR_BATT](#)

Nous avons décidé d'utiliser un AOP en mode comparateur, relié au pont en H par la broche *SLEEP*, qui permet d'activer ou de couper l'alimentation des moteurs.

Le fonctionnement est le suivant :

Grâce à une tension de référence fournie par la batterie, nous obtenons une tension stable servant à comparer avec la tension réelle de la batterie. Pour cela, nous avons choisi le composant LM385-2.5 (figure X), dont la tension maximale est plus élevée que celle du LM385-1.2, ce qui offre une meilleure précision. De plus, son impédance interne plus faible constitue un avantage supplémentaire.

Robot Mini-Sumo (RMS)

FICHE D'AIDE A LA DECISION (FAD)					
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (valeur de 1 à 5)		Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Tension en sortie	Impédance interne		
LM385-2.5		2,5	0,6	25	
		5	5		
LM385-1,2		1,2	1	6	
		2	3		
IUT de Bordeaux	Référence : PRJ_FAD				2 / 7
Département GEII	Révision : 1.6 – 19/09/2022				

Figure 8 : FAD pour le choix de la référence de tension

Ensuite, la batterie et la référence de tension sont toutes deux connectées à l'AOP. Pour ce dernier, nous avons choisi le TL081D (figure X), car il peut être facilement alimenté par le régulateur de tension en 5 V. De plus, sa tension d'entrée est compatible avec les 7,4 V de la batterie. Ce composant répond donc parfaitement à nos attentes. ([Liens vers les FAD](#))

FICHE D'AIDE A LA DECISION (FAD)						
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (valeur de 1 à 5)			Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		tension d'alimentation	tension d'entrée	prix		
LM358ADR		-16V to 16V 4	3V to 32V 0	0,22 5	0	
MCP6002		1,8V to 5,5V 0	0,3V to 6,3V 5	0,24 4	0	
TL081D		-18V to 18 V 5	-15V to 15V 5	0,65 3	75	
IUT de Bordeaux	Référence : PRJ_FAD					2 / 7
Département GEII	Révision : 1.6 – 19/09/2022					

Figure 9 : FAD pour le choix d'amplificateur opérationnel

Grâce à cette configuration, aucun traitement logiciel n'est nécessaire : tout est géré de manière autonome par les composants, ce qui permet une coupure très rapide des moteurs.

Cette solution répond de façon optimale à l'exigence *EXIG_SECUR_BATT*, de manière entièrement autonome, sans intervention ni manipulation externe de notre part. ([Liens vers les FAD](#))

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	20/51
----------------------------------	--	-------

2.1.8 Traitement de l'information

Référence de pré-conception : CPR 9

Exigences client vérifiées par préconception : [EXIG_COMPORTEMENT](#)

Pour ce projet, l'utilisation de la carte Arduino Uno nous a été imposée. Cette carte permet l'acquisition de l'état des capteurs, le traitement des variables correspondant aux capteurs ainsi qu'à la prise de décision en fonction des capteurs, ce qui nous permettra de contrôler le robot.

Référence de pré-conception : CPR 10

Pour garantir la conformité aux exigences établies et formaliser le comportement attendu de notre robot sumo, nous avons réalisé un organigramme qui décrit de manière structurée l'ensemble des fonctions qu'il mettra en œuvre durant les combats.

Robot Mini-Sumo (RMS)

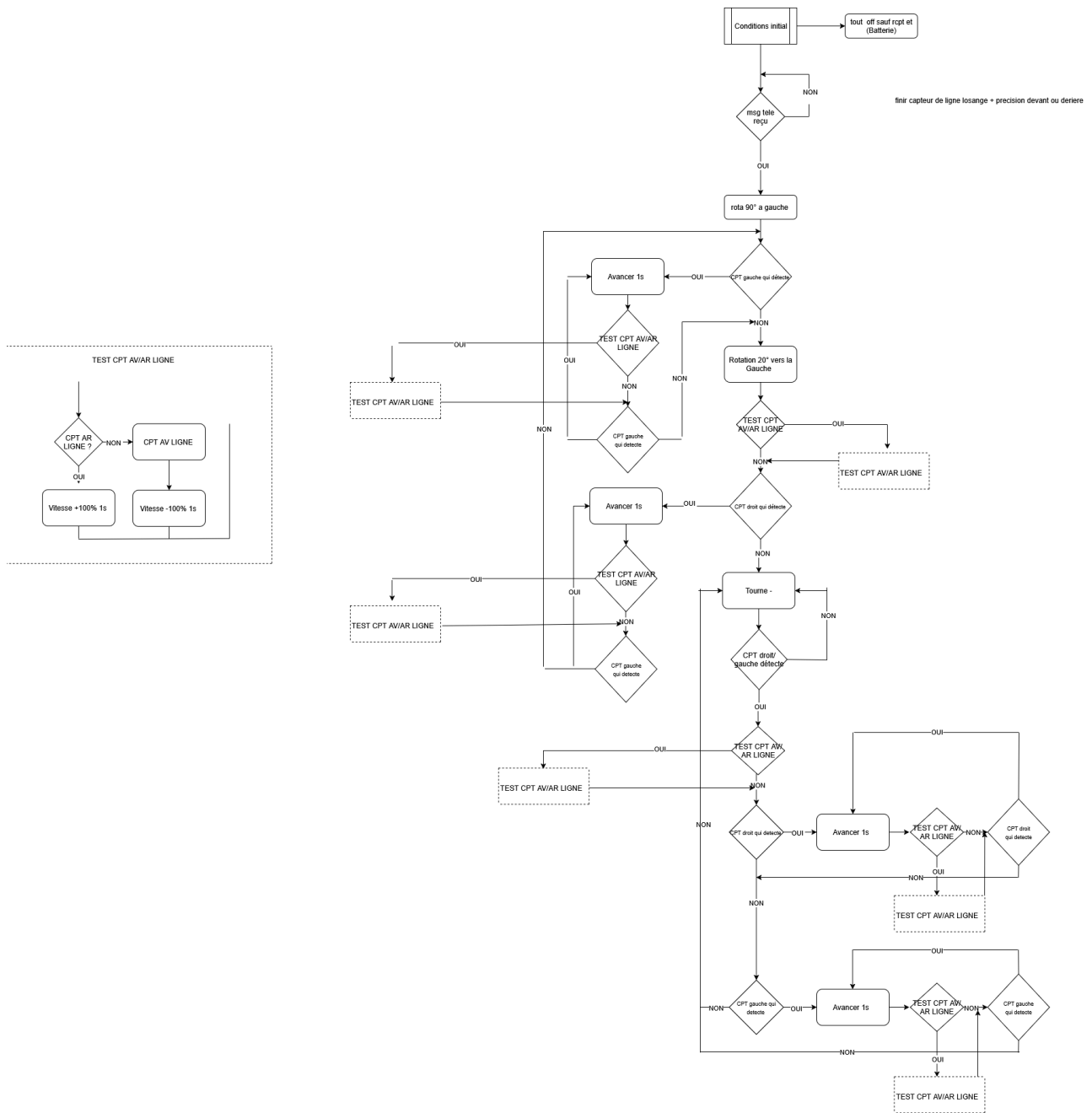


Figure 10 : Algorithme

alorigrammedraw.io (lien pour mieux visualiser l'organigrammes)

2.2 Architecture Informatique

Référence de pré-conception : CPR 11

Exigences client vérifiées par préconception :

EXIG_CHASSIS_DIMENSIONS, EXIG_INTEGRITE_MECA, EXIG_FIXATION_CARTE, EXIG_AUTONOMIE, EXIG_SECUR_BATT, EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_DEPART, EXIG_DEPLACEMENT, EXIG_COMPORTEMENT, EXIG_CARTE

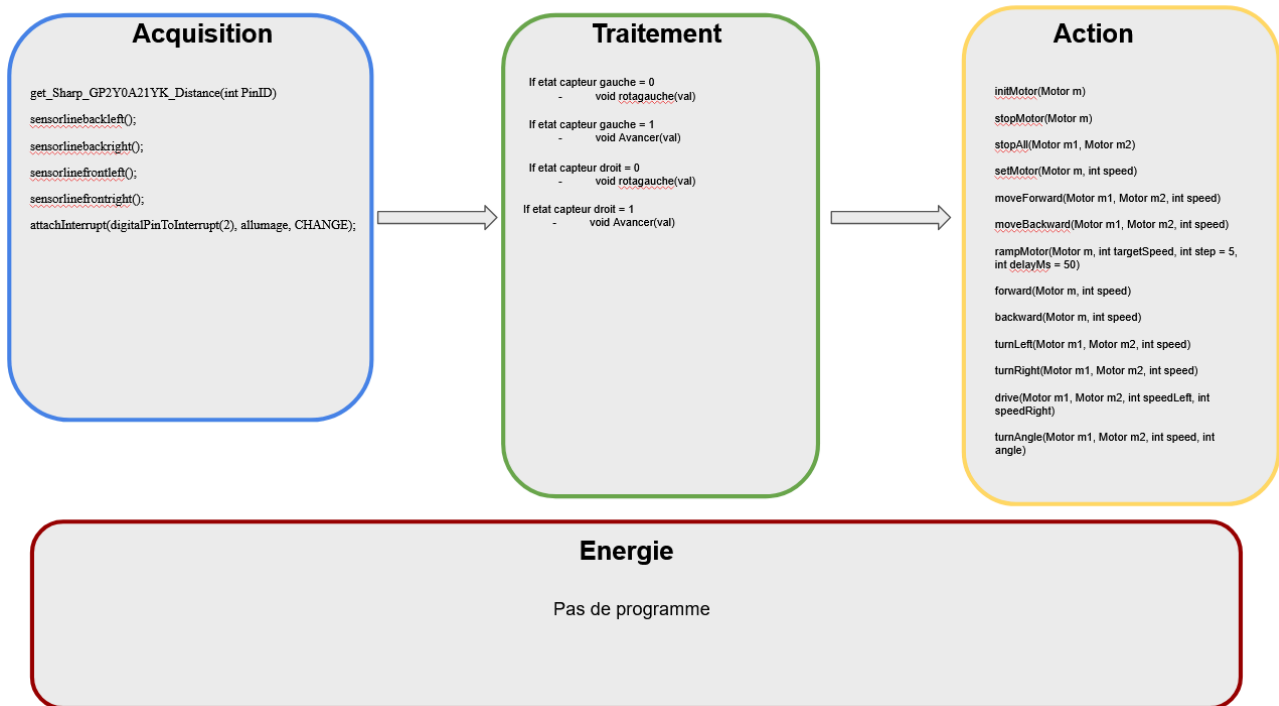


Figure 11 : Schéma de l'architecture informatique

2.2.1 Fonctions d'acquisition

Référence de pré-conception : CPR 12

Exigences client vérifiées par préconception
: [EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_DEPART](#)

Fonction détection d'adversaire (EXIG_ADVERSAIRE) :

- float get_Sharp_GP2Y0A21YK_Distance(int PinID) : Récupère la valeur sur 10 bits du port analogique et calcule la distance de détection du capteur.
- float ADCValue = (float)analogRead(PinID); : Récupère la valeur sur 10 bits du port analogique en décimale pour permettre le calcul de la distance

Fonction détection de bordure d'arène (EXIG_CARTE) :

- int valeurref = 800; : Valeur de référence, si la valeur du capteur de ligne est inférieur à la valeur ref, alors la couleur lu par le capteur de ligne est Blanche sinon la couleur est Noire.
- const int sensorbackleft = A0; : Assignation du port analogique A0 au capteur de ligne arrière gauche.
- const int sensorbackright = A1; : Assignation du port analogique A1 au capteur de ligne arrière droit.
- const int sensorfrontleft = A2; : Assignation du port analogique A2 au capteur de ligne avant gauche.
- const int sensorfrontright = A3; : Assignation du port analogique A4 au capteur de ligne avant droit.
- int valeursensorbackleft; : Valeur sur 10 bits lu sur le port A0 du capteur de ligne arrière gauche.
- int valeursensorbackright; : Valeur sur 10 bits lu sur le port A1 du capteur de ligne arrière droit.
- int valeursensorfrontleft; : Valeur sur 10 bits lu sur le port A2 du capteur de ligne avant gauche.
- int valeursensorfrontright; : Valeur sur 10 bits lu sur le port A3 du capteur de ligne avant droit.
- sensorlinebackleft(); : Récupère la valeur du port analogique correspondant au capteur de ligne arrière gauche, la compare à une valeur de référence. Si la valeur est inférieur à la valeur ref, renvoie la couleur "Blanc".
- sensorlinebackright(); : Récupère la valeur du port analogique correspondant au capteur de ligne arrière droit, la compare à une valeur de référence. Si la valeur est inférieur à la valeur ref, renvoie la couleur "Blanc".
- sensorlinefrontleft(); : Récupère la valeur du port analogique correspondant au capteur de ligne avant gauche, la compare à une valeur de référence. Si la valeur est inférieur à la valeur ref, renvoie la couleur "Blanc".
- sensorlinefrontright(); : Récupère la valeur du port analogique correspondant au capteur de ligne avant droit, la compare à une valeur de référence. Si la valeur est inférieur à la valeur ref, renvoie la couleur "Blanc".

Aucune fonction pour l'exigence EXIG_LUMINOSITE.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	24/51
----------------------------------	--	-------

Fonction détection télécommande (EXIG_DEPART) :

- attachInterrupt(digitalPinToInterrupt(2), allumage, CHANGE); : Interruption sur le port digital 2 correspondant au détecteur IR pour la télécommande pour le lancer le combat.

2.2.2 Fonctions d'action

Référence de pré-conception : CPR 13

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

Pour pouvoir déplacer le robot nous avons décidé de créer une librairie arduino regroupent un grand nombre de fonctions qui peuvent être utiles pour le contrôle du robot. Elles sont toutes détaillées dans la liste ci-dessous (la librairie est déjà prête à l'usage).

1. Fonctions de base

initMotor(Motor m)

Initialise un moteur (définit les broches en sortie et met le moteur à l'arrêt).
À appeler une fois dans `setup()`.

stopMotor(Motor m)

Arrête immédiatement un moteur (PWM = 0).

stopAll(Motor m1, Motor m2)

Arrête les deux moteurs du robot simultanément.

setMotor(Motor m, int speed)

Commande un moteur avec une vitesse entre -255 et +255 :

- `> 0` : marche avant
- `< 0` : marche arrière
- `0` : arrêt

rampMotor(Motor m, int targetSpeed, int step = 5, int delayMs = 50)

Fait varier progressivement la vitesse d'un moteur vers une valeur cible.
Évite les à-coups mécaniques.

forward(Motor m, int speed)

Fait tourner le moteur en marche avant.

backward(Motor m, int speed)

Fait tourner le moteur en marche arrière.

2. Commandes robot (2 moteurs)

moveForward(Motor m1, Motor m2, int speed)

Avancer en ligne droite.

moveBackward(Motor m1, Motor m2, int speed)

Reculer en ligne droite.

turnLeft(Motor m1, Motor m2, int speed)

Tourner sur place vers la gauche (moteur gauche arrière, moteur droit avant).

turnRight(Motor m1, Motor m2, int speed)

Tourner sur place vers la droite (moteur gauche avant, moteur droit arrière).

drive(Motor m1, Motor m2, int speedLeft, int speedRight)

Permet de donner une vitesse indépendante à chaque moteur.

Sert à ajuster la trajectoire (par exemple : 200 à gauche et 150 à droite = virage doux à droite).

3. Commande par angle

turnAngle(Motor m1, Motor m2, int speed, int angle)

Fait tourner le robot d'un angle défini :

- $\text{angle} > 0$ = rotation vers la droite
- $\text{angle} < 0$ = rotation vers la gauche
La durée de rotation est approximée et doit être calibrée expérimentalement.

2.2.3 Fonctions d'énergie

Référence de pré-conception : CPR 14

Exigences client vérifiées par préconception : [EXIG_SECUR_BATT](#), [EXIG_AUTONOMIE](#)

L'exigence de sécurité de la batterie, grâce à la méthode choisie (précisée dans le chapitre [2.1.7](#)), ne nécessite pas de programmation informatique.

2.2.4 Fonctions de traitement

Référence de pré-conception : CPR 15

Exigences client vérifiées par préconception : [EXIG_COMPORTEMENT](#)

Nous allons utiliser les valeurs de variable acquises par les capteurs puis en fonction de celles-ci, nous allons appliquer les fonctions d'action. Pour cela, nous allons utiliser des fonctions IF qui permettront de déterminer l'action à faire en fonction de ce que détectent les capteurs.

Fonction que nous allons utiliser :

If etat capteur gauche = 0

- void rotagauche(val)

If etat capteur gauche = 1

- void Avancer(val)

If etat capteur droit = 0

- void rotagauche(val)

If etat capteur droit = 1

- void Avancer(val)

Robot Mini-Sumo (RMS)

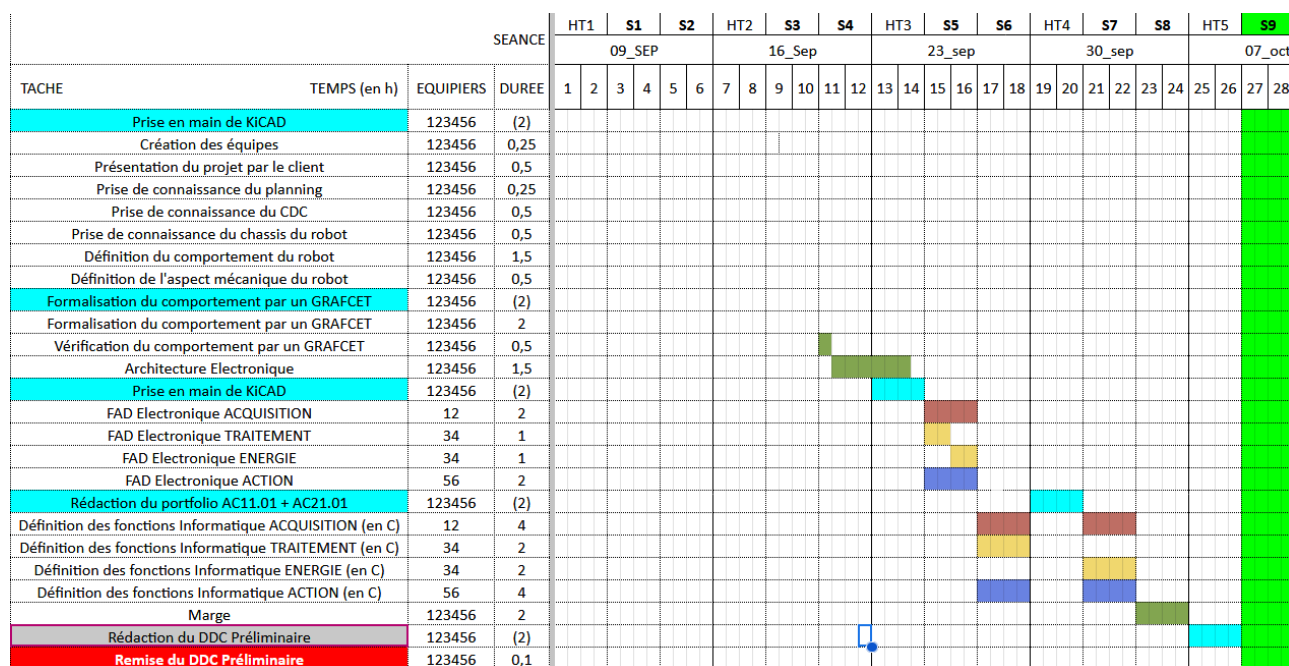


Figure 13 : planning DDC préliminaire

Nous sommes dans le timing du planning nous pouvons donc valider cette exigence.

2.3 Conclusion de la conception préliminaire du produit

Pour conclure, nous avons su identifier une solution technique adaptée, en répondant à l'ensemble des exigences formulées par le client. Afin de faciliter la prise de décision et d'assurer la clarté de nos choix, nous avons également élaboré des fiches d'aide à la décision. Celles-ci permettent de justifier chaque orientation retenue, tout en garantissant une réponse cohérente et structurée aux besoins exprimés.

Robot Mini-Sumo (RMS)

3. Conception détaillée du produit

Ce chapitre détaille la conception du produit développé. Il constitue une preuve de la conformité du produit. Chaque paragraphe de cette étude fait donc clairement référence aux exigences client issues du [CDC].

3.1 Conception détaillée du robot mini-sumo

Chaque bloc fonctionnel doit faire l'objet d'un chapitre de conception détaillé, présenté comme suit.

Référence de conception : CDT_1

Exigences client vérifiées : EXIG_ADVERSAIRE, EXIG_CARTE, EXIG_xxx, EXIG_xxx...

A la suite de la conception préliminaire, l'activité de conception détaillée a été menée. Le schéma électrique complet de la carte est fourni ci-dessous.

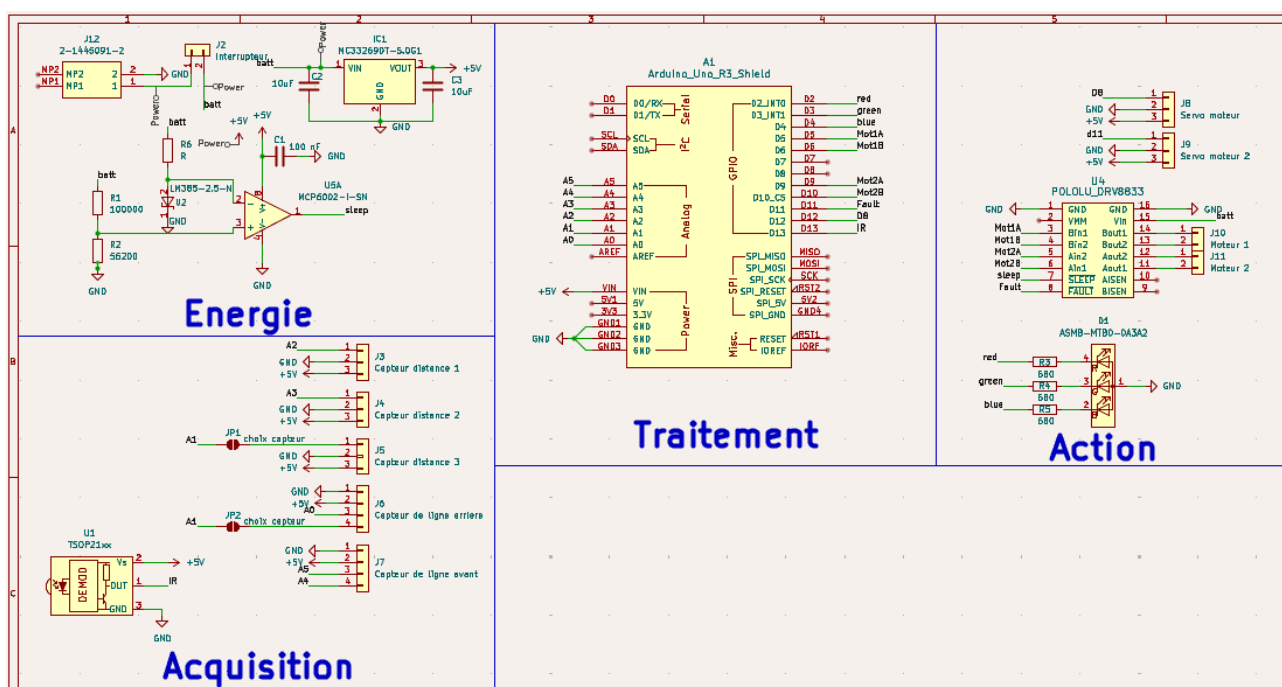


Figure 14 : schéma électrique de la carte robot mini-sumo

3.2 Conception détaillée de la partie acquisition

3.2.1 Capteurs adversaires

Référence de conception : CDT_2

Exigences client vérifiées : EXIG_ADVERSAIRE

3.2.1.1 Schéma électrique des capteurs adversaires

Schéma bloc capteur adversaire :

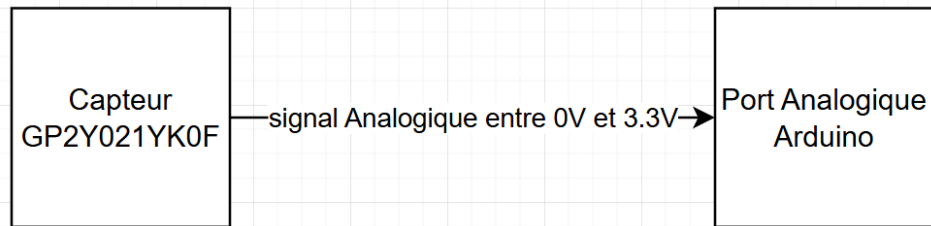


Schéma électronique capteur adversaire :

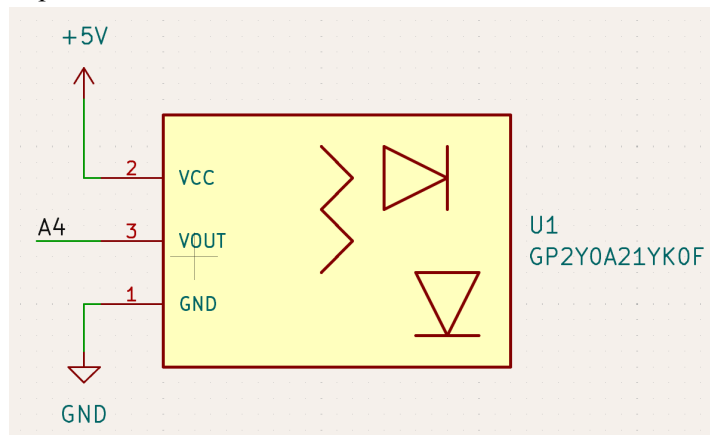


Figure 15 : schéma électrique du capteur d'adversaire

Tension d'entrée donnée par la datasheet : 5V

Intensité donnée par la datasheet : 30mA

Tension théorique de sortie donnée par la datasheet du composant : on récupère une valeur analogique sur 10 entre 0V et 3.3V et on utilise une fonction dans le code pour changer cette valeur sur 10 bits en une valeur de distance.

Robot Mini-Sumo (RMS)

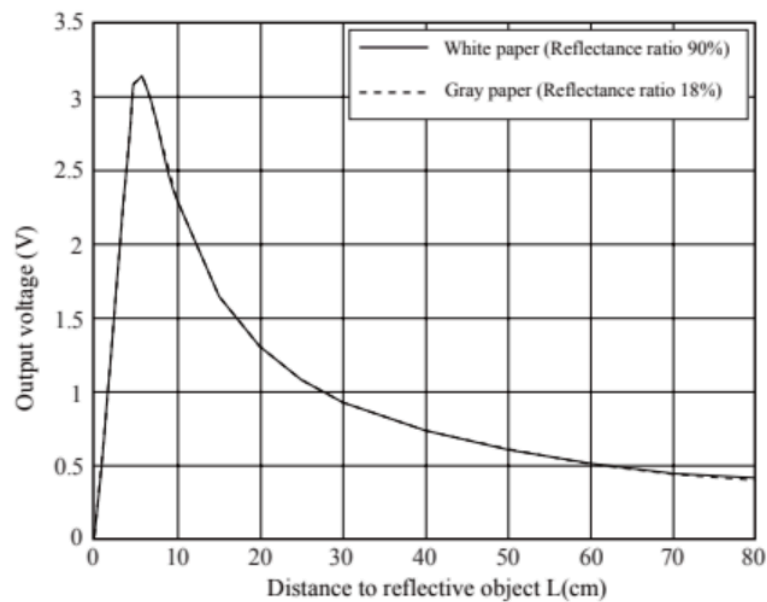


Figure 16 : Datasheet de la tension théorique de sortie du capteur

3.2.1.2 Code informatique des capteurs adversaires

```
const int sensorLeft = A4;    Définition du port A4 correspondant au capteur gauche
const int sensorRight = A5;   Définition du port A5 correspondant au capteur droit

void loop() {
  float distanceLeft = Distance_Adversaire(sensorLeft); Appelle la fonction Distance_Adversaire sur sensorLeft
  float distanceRight = Distance_Adversaire(sensorRight); Idem sur sensorRight

  const float seuil_distance = 45.0; // Seuil en centimètres    Seuil de distance maximum des capteurs

  Suite du code : traitement des données des capteurs
  }

  float Distance_Adversaire(int pinID) {                      Fonction Distance_Adversaire
    float ADCValue = analogRead(pinID);                       ADCValue récupère la valeur du port

    // Formule polynomiale (approximation) → résultat en cm    Formule de distance par rapport à ADCValue
    float d = 200.3775040589502
      - 2.2657665648980 * ADCValue
      + 0.0116395328796 * ADCValue * ADCValue
      - 0.0000299194195 * ADCValue * ADCValue * ADCValue
      + 0.0000000374087 * ADCValue * ADCValue * ADCValue * ADCValue
      - 0.0000000000181 * ADCValue * ADCValue * ADCValue * ADCValue * ADCValue;

    if (d < 0) {
      d = INFINITY;      Si on a une valeur de distance négative, on la transforme en un nombre
    }                    infini pour être supérieur au seuil et ne pas être utilisé dans le traitement des
    return d;            données
  }
}
```

3.2.2 Capteurs de sol

Référence de conception : CDT_3

Exigences client vérifiées : EXIG_CARTE

3.2.2.1 Schéma électrique des capteurs de sol

Schéma bloc capteur de ligne :

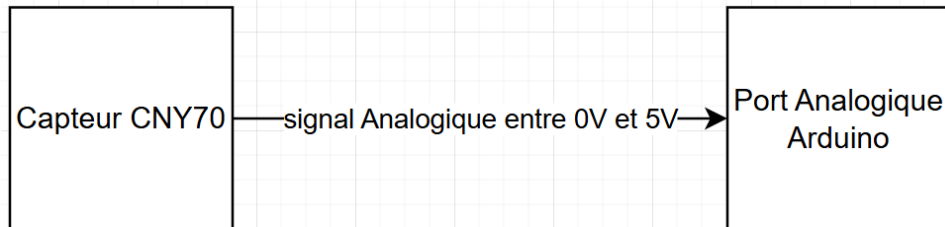


Schéma électronique capteur de ligne :

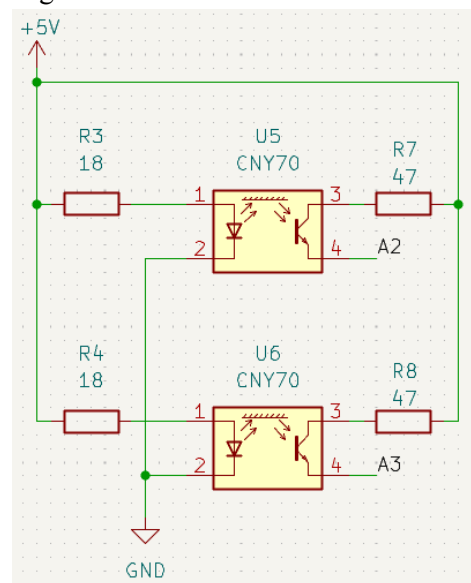


Figure 17 : schéma électrique du capteur de ligne de la carte robot sumo

La valeur de tension de 5V est utilisée pour les capteurs CNY70 car les capteurs adversaires utilisent du 5V.

Led émettrice :

- Intensité : 7.5 mW/sr (milliWatt par stéradian)(stéradian : unité d'angle solide)
- Calcul théorique de résistance : $R = U/I_{max} = (5V - 1.6V) / 0.1A = 34 \text{ Ohms}$

Test pratique une carte fonctionnel avec une résistance de 18 Ohms et une tension de 5V:

Intensité théorique : $I = U/R = 3.4V/18 \text{ Ohms} = 180mA$

Intensité réel de la Led émettrice : 56mA

Une consommation de 56mA par capteur est compatible avec la batterie choisie car le calcul de l'énergie de la batterie a été basé sur la valeur de l'intensité du capteur CNY70 donné par la datasheet (100mA).

3.2.2.2 Code informatique des capteurs de sol

```
//capteurs de ligne
int valeurref = 800;
const int sensorlinebackleft = A0;
const int sensorlinebackright = A1;
const int sensorlinefrontleft = A2;
const int sensorlinefrontright = A3;

int valeursensorlinebackleft;
int valeursensorlinebackright;
int valeursensorlinefrontleft;
int valeursensorlinefrontright;

void loop() {

  sensorlinebackLeft();
  // action
  }
  delay(10);

  sensorlinebackRight();
  //action
  }
  delay(10);

  sensorlinefrontLeft();
  //action
  }
  delay(10);

  sensorlinefrontRight();
  //action
  }
  delay(10);
}

void sensorlinebackLeft(){
  valeursensorbackleft = analogRead(sensorbackleft);
}

void sensorlinebackRight(){
  valeursensorbackright = analogRead(sensorbackright);
}

void sensorlinefrontLeft(){
  valeursensorfrontleft = analogRead(sensorfrontleft);
}

void sensorlinefrontRight(){
  valeursensorfrontright = analogRead(sensorfrontright);
}
```

Valeur de seuil : inférieur = couleur détecté Blanc et supérieur couleur détecté Noir

Définition du port A0 correspondant au capteur de ligne arrière gauche

Définition du port A1 correspondant au capteur de ligne arrière droit

Définition du port A2 correspondant au capteur de ligne avant gauche

Définition du port A3 correspondant au capteur de ligne avant droit

Définition d'une variable int pour la valeur du port A0

Définition d'une variable int pour la valeur du port A1

Définition d'une variable int pour la valeur du port A2

Définition d'une variable int pour la valeur du port A3

Dans la boucle

Appelle de la fonction sensorlinebackLeft

Emplacement traitement des données du capteur de ligne arrière gauche

Délai pour fluidifier le code

Appelle de la fonction sensorlinebackRight

Emplacement traitement des données du capteur de ligne arrière droit

Délai pour fluidifier le code

Appelle de la fonction sensorlinefrontLeft

Emplacement traitement des données du capteur de ligne avant gauche

Délai pour fluidifier le code

Appelle de la fonction sensorlinefrontRight

Emplacement traitement des données du capteur de ligne avant droit

Délai pour fluidifier le code

Fonction sensorlinebackLeft

valeursensorbackleft récupère la valeur du port A0

Fonction sensorlinebackRight

valeursensorbackleft récupère la valeur du port A1

Fonction sensorlinefrontLeft

valeursensorbackleft récupère la valeur du port A2

Fonction sensorlinefrontRight

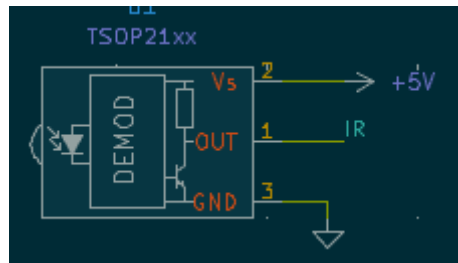
valeursensorbackleft récupère la valeur du port A3

3.2.3 Capteurs de télécommande

Référence de conception : **CDTxx**

Exigences client vérifiées : **EXIG_DEPART**

3.2.3.1 Schéma électrique des capteurs de télécommande



On alimente le capteur infrarouge en 5V et on connecte la broche de donnée à l'arduino
Le TSOP21 fonctionne avec une large zone de réception et etc

3.2.3.2 Code informatique des capteurs adversaires

```
//Library
#include "IRremote.h"

//Variable
int receiverPin = 13; // Le Capteur de la télécommande est sur l'entrée 13
int flag=0;
int info;
IRrecv irrecv(receiverPin);

void initIR(){
  irrecv.enableIRIn(); // Start the receiver
}

int lireIR(){
  int flag = 0;
  static long temps =0;
  if(IrReceiver.decode()>0){
    if((millis()-temps)>200){
      flag=1;
    }
    temps = millis();
    irrecv.resume();
  }
  return flag;
}

void setup(){
  initIR();
  pinMode(5, OUTPUT); // Permet de mettre une LED sur la sortie 5
}
```

Robot Mini-Sumo (RMS)

```
void loop() {  
  info = lireIR();  
  if(info==1){  
    Etat_LED = 1 - Etat_LED;  
  }  
  digitalWrite(12, Etat_LED);  
}
```

Dans ce code d'essai des capteurs infrarouges, nous faisons appel à une fonction setup, loop, et à deux fonctions qui sont : void initIR() et int lireIR(). Nous avons utilisé la bibliothèque IRremote.h. Cette dernière permet d'utiliser différentes fonctions afin de recevoir une donnée de notre télécommande.

Voici les différentes fonctions utilisées :

- initIR() : cette fonction fait appel à une fonction de la bibliothèque IRremote.h qui est irrecv.enableIRIn(). Cette dernière permet d'activer le capteur infrarouge afin d'être prêt à recevoir une donnée de la télécommande.
- Notre fonction lireIR() contient une autre fonction de la bibliothèque IRremote.h : IrReceiver.decode(). Celle-ci a pour but de décrypter les informations envoyées par notre télécommande.
- irrecv.resume() : cette fonction issue également de la bibliothèque IRremote.h permet d'actualiser les valeurs stockées afin d'être prêt à la réception d'une nouvelle donnée.

La LED est éteinte quand l'état de la LED (Etat_LED) vaut 0.

Dans notre fonction, nous utilisons la fonction millis() afin d'être le plus réactif à l'appui sur la télécommande. En effet, la condition millis()-temps>200 (ici le temps correspond au temps de l'appui précédent, nous remettons à jour le temps avec la valeur millis()), nous permet de faire appel à la fonction irrecv.resume() dès l'appui. Nous n'avons pas besoin d'attendre le relâchement du bouton. A chaque appui, l'état logique de la LED s'inverse.

Pour conclure, ce code permet de vérifier si le récepteur infrarouge et à quel moment il exécute nos lignes de code avec l'allumage de notre LED. La LED remplace l'état des moteurs pour les besoins du test et représente si les moteurs sont en fonctionnement ou en arrêt.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	37/51
----------------------------------	--	-------

3.3 Conception détaillée de la partie action

3.3.1 Pont en H

Référence de conception : **CDTI**

Exigences client vérifiées : EXIG_DEPLACEMENT, EXIG_CARTE, EXIG_COMPOTEMENT

3.3.1.1 Schéma électrique du pont en H

Avec les ponts en H, il n'a pas été nécessaire de dimensionner ou de normaliser les valeurs, la conception répondait déjà aux exigences sans ajustement supplémentaire.

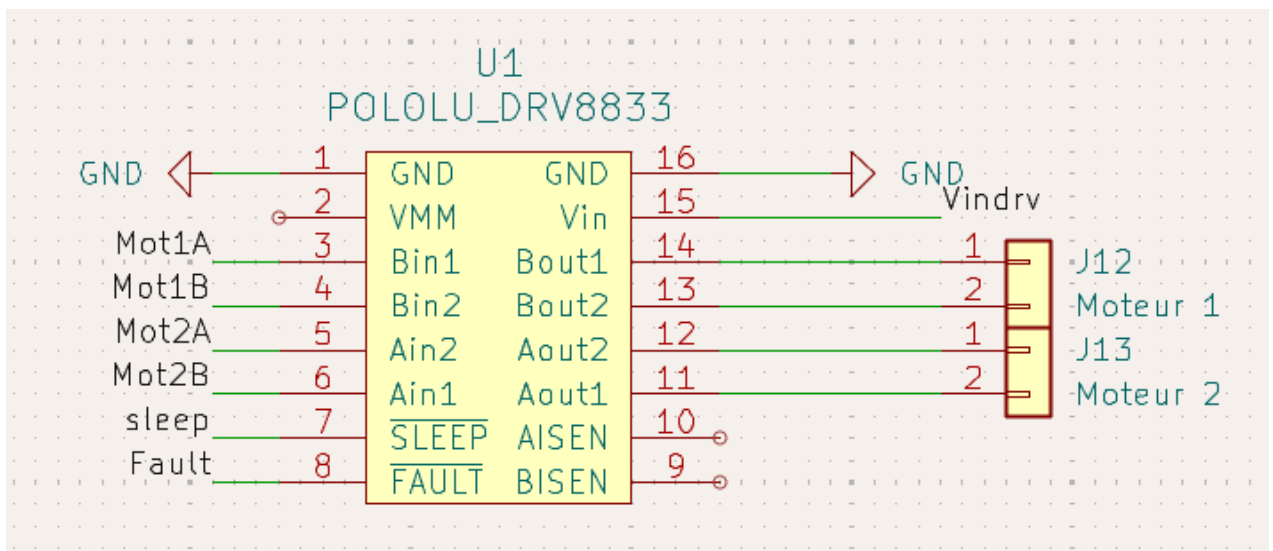


Figure18: Schéma électrique du pont en H

3.3.1.2 Code informatique du pont en H

Afin de répondre aux exigences du CDC concernant la partie action du robot Mini-Sumo (EXIG_DEPLACEMENT, EXIG_COMPOTEMENT et EXIG_CARTE), nous avons créé une bibliothèque dédiée, nommée MotorLib.h. Cette bibliothèque regroupe l'ensemble des fonctions nécessaires au pilotage des moteurs à courant continu à l'aide d'un pont en H.

Elle a pour objectif de simplifier le contrôle du déplacement du robot tout en assurant une structure de code claire, modulaire et réutilisable. Voici le lien vers le [Code Informatique](#).

Dans le fichier MotorLib.h, chaque moteur est défini par une structure de données :

```
struct Motor {
    int pin1;
    int pin2;
```

```
};
```

Cette structure permet d'associer à chaque moteur les deux sorties PWM nécessaires à son pilotage. Le code reste ainsi générique et modulaire, ce qui facilite le contrôle simultané de plusieurs moteurs.

Initialisation et arrêt du pont en H

La fonction ci-dessous configure les deux broches du moteur comme sorties numériques afin de pouvoir transmettre les signaux de commande au pont en H. Une fois l'initialisation effectuée, le moteur est immédiatement arrêté par un appel à `stopMotor()` pour éviter toute rotation involontaire lors de la mise sous tension.

```
void initMotor(Motor m) {
    pinMode(m.pin1, OUTPUT);
    pinMode(m.pin2, OUTPUT);
    stopMotor(m);
}
```

Ensuite, on a mis les deux entrées du pont en H sont placées à zéro, ce qui coupe complètement le courant traversant le moteur. Le moteur se trouve alors en roue libre.

```
void stopMotor(Motor m) {
    analogWrite(m.pin1, 0);
    analogWrite(m.pin2, 0);
}
```

Cette commande permet de garantir un arrêt sûr, sans court-circuit interne au pont en H.

La fonction `stopAll(Motor m1, Motor m2)` applique ce même principe simultanément sur les deux moteurs du robot :

```
void stopAll(Motor m1, Motor m2) {
    stopMotor(m1);
    stopMotor(m2);
}
```

Commande simple d'un moteur à l'aide du pont en H

La fonction principale de pilotage est la suivante :

```
void setMotor(Motor m, int speed) {
    if (speed > 0) {
        analogWrite(m.pin1, constrain(speed, 0, 255));
        analogWrite(m.pin2, 0);
    } else if (speed < 0) {
        analogWrite(m.pin1, 0);
        analogWrite(m.pin2, constrain(-speed, 0, 255));
    }
}
```

```
} else {
  stopMotor(m); }
```

Le pont en H comprend quatre transistors commandés par les signaux **pin1** et **pin2** :

⇒ Lorsque **pin1** est alimentée et **pin2** à 0, le courant traverse le moteur dans un sens donné → rotation avant.

⇒ Lorsque **pin1** est à 0 et **pin2** alimentée, le courant circule dans le sens inverse → rotation arrière.

⇒ Lorsque les deux signaux sont à 0, aucun courant ne circule → arrêt du moteur.

La valeur transmise à **analogWrite()** correspond à une valeur PWM comprise entre 0 et 255, représentant le rapport cyclique appliqué au pont en H. Cette modulation permet de régler la vitesse du moteur proportionnellement à la consigne **speed**. Ainsi, **setMotor()** constitue le cœur logiciel du pilotage du pont en H : elle traduit directement une consigne de vitesse (positive ou négative) en tension moyenne aux bornes du moteur.

Accélération et décélération progressive

```
void rampMotor(Motor m, int targetSpeed, int step, int delayMs) {
  static int currentSpeed = 0;

  if (targetSpeed > currentSpeed) {
    for (int s = currentSpeed; s <= targetSpeed; s += step) {
      setMotor(m, s);
      delay(delayMs);
    }
  } else {
    for (int s = currentSpeed; s >= targetSpeed; s -= step) {
      setMotor(m, s);
      delay(delayMs);
    }
  }
  currentSpeed = targetSpeed;
}
```

Cette fonction crée une rampe de variation progressive entre la vitesse actuelle et la vitesse cible. Elle appelle successivement **setMotor()** avec des valeurs intermédiaires afin d'éviter des transitions trop brutales dans le pont en H. Cette approche protège les moteurs et transistors contre les pics de courant, tout en assurant un déplacement fluide du robot.

Commandes combinées pour les deux ponts en H

Comme le robot est équipé de deux moteurs, chacun commandé par son propre pont en H., les fonctions suivantes coordonnent leur fonctionnement pour réaliser différents mouvements.

Avancer

```
void moveForward(Motor m1, Motor m2, int speed) {
  setMotor(m1, abs(speed));
```

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	40/51
----------------------------------	--	-------


```
setMotor(m2, abs(speed));  
}
```

Les deux ponts en H reçoivent des signaux identiques (courant dans le même sens), permettant un mouvement rectiligne avant.

Reculer

```
void moveBackward(Motor m1, Motor m2, int speed) {  
    setMotor(m1, -abs(speed));  
    setMotor(m2, -abs(speed));  
}
```

Le courant est inversé sur les deux moteurs, produisant un déplacement arrière.

Tourner à gauche / droite

```
void turnLeft(Motor m1, Motor m2, int speed) {  
    setMotor(m1, -abs(speed));  
    setMotor(m2, abs(speed));  
}
```

```
void turnRight(Motor m1, Motor m2, int speed) {  
    setMotor(m1, abs(speed));  
    setMotor(m2, -abs(speed));  
}
```

Chaque moteur reçoit une consigne opposée :

⇒ un moteur tourne vers l'avant,
⇒ l'autre vers l'arrière.

Les deux ponts en H fonctionnent alors en opposition, ce qui provoque une rotation sur place du robot.

Commande fine et rotation angulaire

Contrôle différentiel

```
void drive(Motor m1, Motor m2, int speedLeft, int speedRight) {  
    setMotor(m1, speedLeft);  
    setMotor(m2, speedRight);  
}
```

Cette fonction permet de commander séparément les deux ponts en H. Elle est utilisée pour des ajustements de trajectoire ou pour des stratégies plus fines (suivi de ligne, poursuite d'adversaire).

Rotation angulaire approximative

```
void turnAngle(Motor m1, Motor m2, int speed, int angle) {  
    float k = 8.5;  
    int duration = abs(angle) * k * (150.0 / speed);  
  
    if (angle > 0) {  
        turnRight(m1, m2, speed);  
    } else {  
        turnLeft(m1, m2, speed);  
    }  
    delay(duration);  
    stopAll(m1, m2);  
}
```

Cette fonction calcule une durée proportionnelle à l'angle souhaité, puis fait tourner les deux ponts en H en sens opposé pendant ce temps. Le robot pivote ainsi d'un angle approximatif. La constante k (en millisecondes par degré) est déterminée expérimentalement en fonction de la vitesse PWM et du diamètre des roues.

De cette façon, l'ensemble du code de la bibliothèque MotorLib traduit les consignes logicielles de mouvement en signaux PWM et logiques applicables aux ponts en H.

Chaque pont reçoit deux signaux :

- ⇒ un signal PWM proportionnel à la vitesse,
- ⇒ un niveau logique déterminant le sens de rotation.

Ce code informatique assure :

- ⇒ une commande bidirectionnelle complète du moteur,
- ⇒ une maîtrise fine de la vitesse,
- ⇒ une coordination parfaite entre les deux ponts en H pour réaliser les mouvements du robot Mini-Sumo.

Pour conclure, le code informatique du pont en H, contenu dans la bibliothèque MotorLib, constitue le cœur du système d'action du robot. Il assure aussi le lien entre la carte Arduino et les moteurs à courant continu en traduisant les consignes logicielles en tensions modulées et polarisées sur les bornes des moteurs. Cette implémentation respecte les exigences du CDC tout en offrant un pilotage robuste, évolutif et modulaire, indispensable au bon fonctionnement du robot Mini-Sumo.

3.4 Conception détaillée de la partie énergie

3.4.1 Pont diviseur de tension

Référence de conception : CDT1

Exigences client vérifiées : EXIG_SECUR_BATT

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	42/51
----------------------------------	--	-------

3.4.1.1 Schéma électrique du pont diviseur de tension

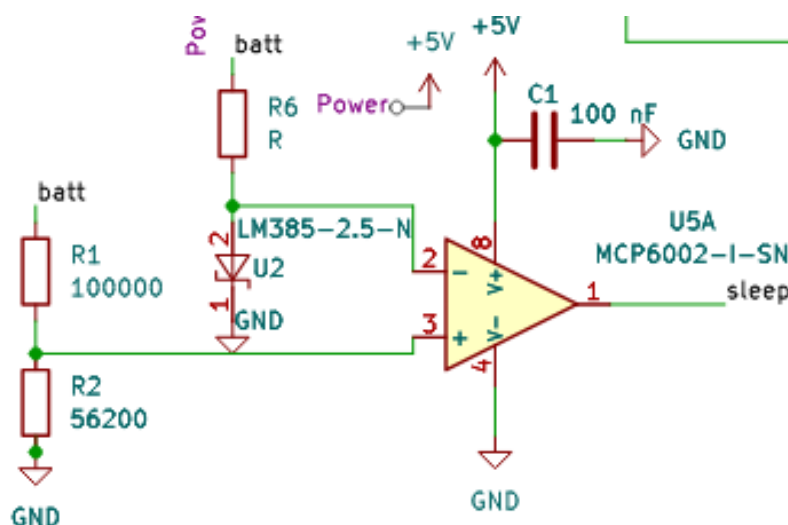


Figure 19: Schéma du pont diviseur de tension avec le comparateur

Afin d'assurer le respect de l'exigence de sécurité batterie nous utilisons un pont diviseur de tension et une référence de tension connecté à un comparateur. La référence de tension est fixée à 2,5 V, nous calculons alors les valeurs de résistance du pont diviseur de tension afin d'avoir 2,5 V en sortie lorsque la tension d'entrée (tension de la batterie vaut 6,7 V). Afin de trouver les valeurs de résistance adapté, nous utilisons la relation du diviseur de tension ($V = \frac{UR_2}{R_1 + R_2}$) en isolant R2, ainsi nous avons $R_2 = \frac{V_{out}R_1}{U_{batt} - V_{out}}$. Nous fixons R1 = 100 kΩ afin de limiter le courant traversant le pont diviseur. Nous calculons donc R2 = 59482,7 Ω. Après normalisation de la valeur, nous sélectionnons

3.4.1.2 Code informatique du pont diviseur de tension

Grâce à la méthode choisie (pont diviseur de tension, référence de tension et comparateur), nous n'avons pas besoin d'utiliser de code.

3.4.2 régulateur de tension

Référence de conception : CDT2

Exigences client vérifiées : Pas de lien direct avec une exigence mais obligatoire pour le fonctionnement de la carte

3.4.2.1 Schéma électrique du régulateur de tension

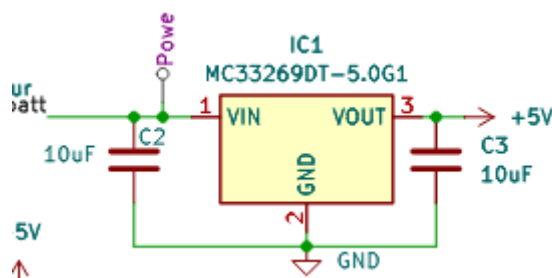


Figure 20 :Schéma électrique du régulateur de tension

Le régulateur de tension est le même que celui choisi en conception préliminaire.

3.4.3 Autonomie

Référence de conception : CDT3

Exigences client vérifiées : EXIG_AUTONOMIE

Afin de contrôler le courant consommé par le circuit nous allons intégrer un ampèremètre à l'entrée du circuit électrique puis nous allons tester d'abord sans les moteurs allumés puis ensuite avec les moteurs allumés, puis nous pouvons en conclure via calcul si la batterie choisie en conception préliminaire est suffisante.

Rappel : Batterie Lipo 2S 7.4V 800mAh

Nous reprenons la vérification théorique en corrigeant les calculs, en effet la consommation des capteurs de ligne a été mesuré avec un ampèremètre, et nous avons ajouté certain composant nous calculons donc la consommation :

- 2 motoréducteurs (133.92mA)
- 4 CNY70 (224mA)
- 2 GP2Y0A21YKOF (60 mA)
- 1 Arduino (50mA)
- 2 DRV 8833 (3.4 mA)
- 1 récepteur infrarouge (3 mA)
- Pont Divsiseur de tension (0,047mA)
- AOP (0,1mA)
- Régulateur linéaire (20mA)

Total : 494,5 mA

Nous calculons alors la capacité minimale de la batterie : $E = i \times t$

$$E = 494,5 \times 1.08 = 534,06 \text{ mAh}$$

Nous prenons une marge de 20% donc $E = 534,6 \times 1.2 = 641,52 \text{ mAh} < 800 \text{ mAh}$

3.4.3.1 Schéma électrique

3.5 Conception détaillée de la partie traitement

3.5.1 Le microcontroleur ATMEGA328P-PU

Référence de conception : CDT1

Exigences client vérifiées : EXIG_COMPORTEMENT

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	44/51
----------------------------------	--	-------

3.4.1.1 Schéma électrique du microcontrôleur

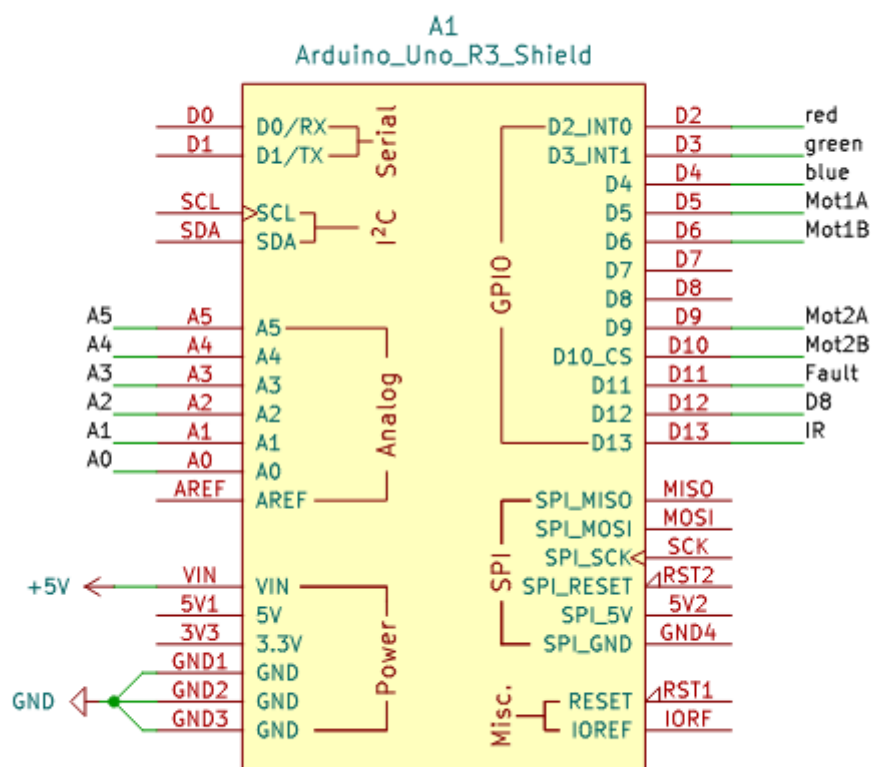


Figure 21: cablage arduino

3.3.1.2 Code informatique du microcontrôleur

Fonction utilisé pour la programmation du microcontrôleur :

Fonction de détections de l'émission infrarouge :

```
while(IrReceiver.decode() == false){
  digitalWrite(Vpin, HIGH);
}
```

Dans le voidsetup, nous mettons une boucle while qui reste active tant qu'il n'y a pas de signal infrarouge reçu. La fonction IrReceiver.decode renvoie false s'il n'y a pas de signal reçu et true s'il y a un signal. Nous mettons la boucle while à la fin du voidsetup afin qu'elle ne s'exécute qu'une seule fois au début du combat. Le robot démarrera dès que le juge appuie sur la télécommande, comme indiqué dans le cahier des charges. La commande digitalWrite(Vpin, HIGH); sert à allumer une LED de débogage, elle permet de savoir si la carte est alimentée.

Détection de ligne :

```
if(valeurBL < valeurrefR){ // ligne a l'arriere gauche
  moveForward(leftMotor, rightMotor, 200); //Avancer(moteur 100%) 1s
  delay(1000);
  stopAll(leftMotor, rightMotor);
}
```

Robot Mini-Sumo (RMS)

Afin de s'assurer que le robot réagisse bien en fonction des lignes du dohyo, nous utilisons une fonction IF avec une comparaison entre la valeur lue par le capteur et une valeur de référence. Le dohyo est noir, les lignes qui l'entourent sont blanches, lorsque le capteur est sur du blanc sa valeur diminue sous la référence. S'il y a détection de la ligne à l'arrière, nous faisons avancer le robot (fonction : moveForward), si il y a détection de ligne à l'avant, nous faisons reculer le robot (fonction : moveBackward).

Détection d'adversaire :

```
if (distanceLeft < seuil_distance && distanceRight > seuil_distance){ // adversaire a gauche-->on avance vers la gauche
    drive(leftMotor, rightMotor, 200, 100);
}
```

Pour la détection d'adversaire, nous mettons une succession de IF, pour faire face à toutes les possibilités (adversaire détecté par le capteur gauche, droit, par les deux capteurs ou aucun capteur) en fonction de l'endroit où est détecté l'adversaire, nous prenons les décisions adéquates. Si l'adversaire est détecté par les deux capteurs, il avance. S'il est détecté par le capteur droit ou gauche, il avance en effectuant une rotation pour suivre l'adversaire. La fonction drive permet de faire cela en commandant indépendamment la vitesse du moteur droit et celle du moteur gauche. Si il n'y a pas de détection d'adversaire, le robot fait une rotation de 90° sur la gauche jusqu'à détections du robot adversaire (fonction : turnleft). Nous mettons la fonction detectionligne à la fin de la fonction detectionadversaire, la fonction detectionlignes correspond à la fonction décrite dans le paragraphe détection de ligne

Référence de pré-conception : CPR 15

Exigences client vérifiées par préconception : [EXIG_COUT](#)

Les coûts n'ont pas changés par rapport au ddc préliminaire

Référence de pré-conception : CPR 16

Exigences client vérifiées par préconception : [EXIG_DELAI](#)

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ4 Révision : 1 – 29/09/2025	46/51
----------------------------------	--	-------

Robot Mini-Sumo (RMS)

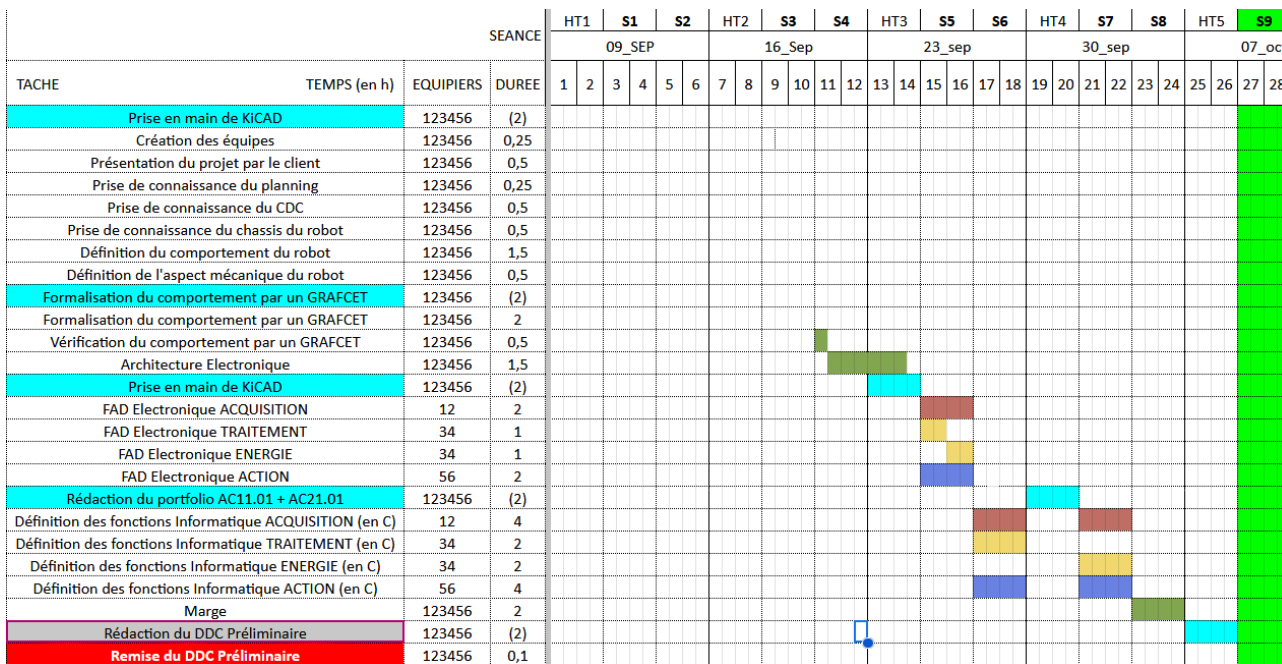


Figure 13 : planning DDC préliminaire

Nous sommes dans le timing du planning nous pouvons donc valider cette exigence.

4. Dérivage des solutions techniques retenues

4.1 Test driver

Référence de la simulation : SIM<1>

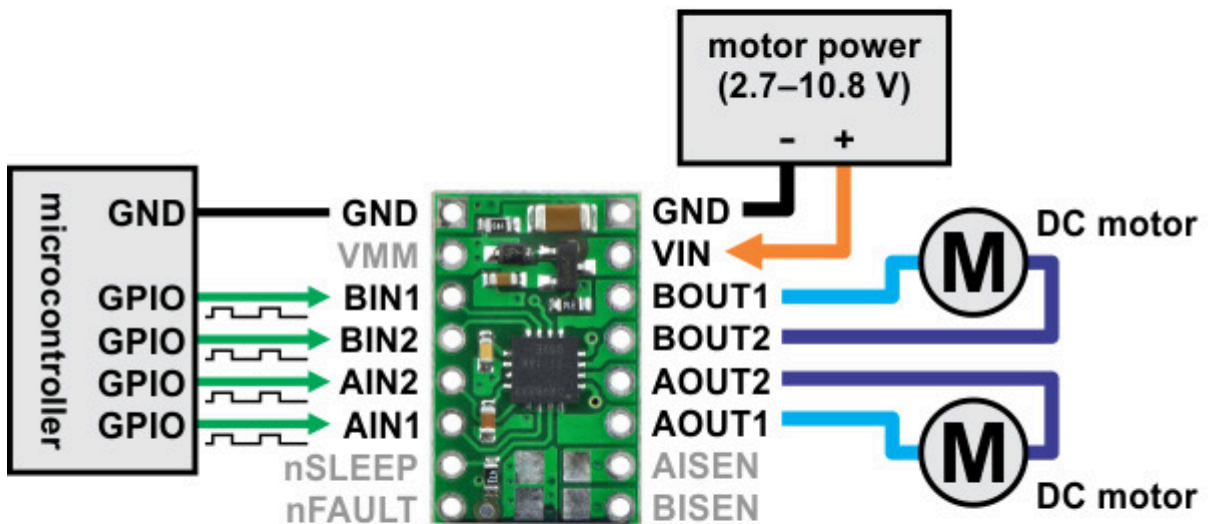
Exigences client vérifiées : EXIG_DEPLACEMENT

But de l'essai : vérifier si les les moteur tourne bien avec les drivers

Fichiers DRV8833

Procédure de simulation :

On câble les moteur la carte arduino et l'alimentation de table au drv8833 selon le schéma donné par le constructeur:



On envoie ensuite un programme qui fait tourner les moteur dans les deux sens tout en modifiant la vitesse en l'augmentant et la diminuant progressivement

Résultats attendus :

On attend des movement moteur correspondant au commande envoyé depuis l'arduino

Résultats obtenus :

Le moteurs tourne bien dans le sens attendue au vitesse attendu et nous permet bien de controller les moteur de façon indépendante ou synchroniser

Statut de l'essai : Conforme

Problèmes rencontrés :

Les câbles de l'alimentation de table on gné quand nous avons voulu faire bouger le robot pour tester des mouvement nous avons donc fait marcher le robot grâce à des pile 9V

4.2 Test capteur de distance

Référence de la simulation : SIM<2>

Exigences client vérifiées : EXIG_ADVERSAIRE

But de l'essai : Tester le bon fonctionnement des capteurs de distance

Fichiers :  code informatique

Procédure de simulation :

On connecte le capteur de distance et on le teste avec un programme qui est censé nous renvoyer les valeurs de distances en centimètres, on place le capteur face à une règle et on éloigne un bloc représentant l'ennemi de centimètre en centimètre en comparant les valeurs de la règle à celle renvoyés par le programme.

Résultats attendus :

Les valeurs de la règle et du programme sont identiques.

Résultats obtenus :

La distance est bien renvoyée de façon plus ou précise jusqu'à 3cm d'écart dans les mesures pas gênant pour notre utilisation

Statut de l'essai : conforme

Problèmes rencontrés

La librairie donnée par le constructeur n'était pas fiable nous avons donc utilisé le programme d'une personne ayant relevé une courbe des distances du capteur de distance avant de faire une régression linéaire pour obtenir une formule renvoyant la valeur en centimètre selon la distance.

4.3 Conclusion de la simulation / prototypage rapide du produit

Les simulations sont conforme aux exigence

5. Conclusion de la conception du produit

En conclusion l'ensemble des exigence sont conforme aux attente pas de problème à passer à la fabrication

6. Matrice de conformité du produit

Ce chapitre synthétise par l'intermédiaire d'un tableau la conformité du produit développé par rapport aux exigences issues du Cahier des Charges.

Exigence	Méthodes Vérification	Statut
EXIG_ENERGIE	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_CHASSIS_DIMENSIONS	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_MASSE	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_INTEGRITE_MECA	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_FIXATION_CARTE	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_AUTONOMIE	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_SECUR_BATT	Conception Dériskage Conception/Fab. Vérification	Conf.

Robot Mini-Sumo (RMS)

EXIG_ADVERSAIRE	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_LUMINOSITE	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_DEPART	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_DEPLACEMENT	Conception Dériskage Conception/Fab. Vérification	Conf.
EXIG_COMPORTEMENT	Conception Dériskage Conception/Fab. Vérification	Conf.